

MedlyPharma

INTRODUCTION:

Medley Pharma is a versatile and user-friendly software tool designed to streamline the creation and management of online forms. Suitable for surveys, registrations, feedback collection, and more, it offers a wide range of customizable form fields—including text inputs, checkboxes, dropdowns, and file uploads—that can be arranged using a simple drag-and-drop interface. Users can fully tailor the appearance of forms to match their branding, while robust security features like CAPTCHA, SSL encryption, and submission limits help protect data integrity. Medley Pharma also integrates seamlessly with third-party applications to automate workflows and notifications. With built-in data visualization tools, users can generate reports and export responses for deeper analysis. Its flexibility, ease of use, and powerful features make it an ideal choice for businesses, educators, event organizers, and researchers seeking an efficient form-building solution.

SCENARIO Based Case Study:

Meet Alex, a pharmaceutical sales representative with a demanding schedule who values efficiency and organization in his work. Alex is responsible for promoting and selling a range of pharmaceutical products to healthcare professionals, which requires him to stay updated on product information, manage customer relationships, and track sales activities.

Alex discovers Medly Pharma, a comprehensive web application designed to streamline pharmaceutical sales operations. Intrigued by its features, Alex decides to explore how Medly Pharma can help him enhance his productivity and effectiveness.

User Registration and Authentication: Alex registers an account on Medly Pharma, providing his professional details and preferences. He logs in securely using his credentials, ensuring that his data remains private and secure.

Product Catalog Management: Alex explores Medly Pharma's extensive product catalog, which includes detailed information about pharmaceutical products, such as indications, dosages, and availability. He can easily access this information to educate healthcare professionals and answer their queries effectively.

Customer Relationship Management (CRM): Medly Pharma offers a CRM system that allows Alex to manage his interactions with healthcare professionals efficiently. He can track customer communications, schedule follow-up appointments, and maintain detailed customer profiles to personalize his approach.

Sales Reporting and Analytics: Medly Pharma provides Alex with tools to track his sales activities and performance metrics. He can generate reports, analyze trends, and identify opportunities for growth, helping him make informed decisions to drive sales.

Order Management: Alex can place orders for pharmaceutical products directly through Medly Pharma, streamlining the ordering process and ensuring timely delivery to his customers.

Training and Support: Medly Pharma offers training materials and support resources to help Alex maximize his use of the platform. He can access tutorials, FAQs, and customer support to resolve any issues or queries he may encounter.

Alex's Experience: Thanks to Medly Pharma, Alex can now manage his pharmaceutical sales activities more efficiently and effectively. He can focus on building strong relationships with healthcare professionals, promoting pharmaceutical products, and driving sales growth. Medly Pharma has become an indispensable tool in Alex's work, helping him achieve his professional goals with ease and confidence.

SYSTEM REQUIREMENTS:

Software Requirements:

To ensure smooth development, deployment, and usage of the **MedlyPharma** Web-Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable agricultural platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above):** Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing. 🖱️ [Download Node.js](#)
- **npm (v8 or above):** A package manager required to install dependencies for React and related libraries.
- **React.js:** A JavaScript library for building dynamic and responsive user interfaces.
- **Browser:** Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- **Express.js:** A lightweight web framework for building RESTful APIs.
- **MongoDB:** A NoSQL database used to store structured and unstructured data related to farms, users, and resources. 🖱️ [Download MongoDB](#)
- **Postman:** Tool for testing APIs during development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.

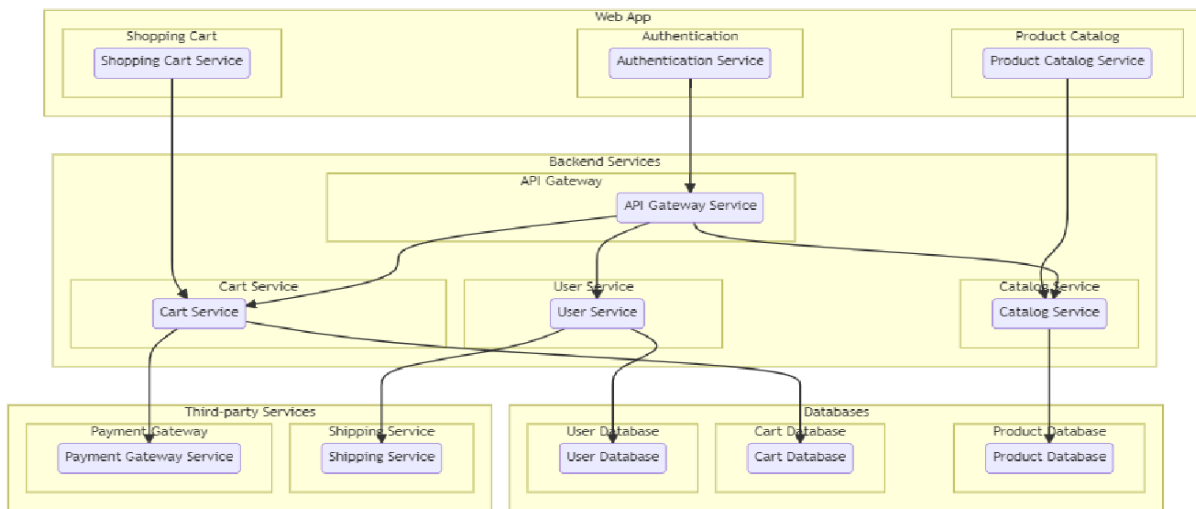
2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage:** At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- **Display:** 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

PROJECT ARCHITECTURE:

TECHNICAL ARCHITECTURE:



In this architecture, the flow begins with a user (represented by the "User" node) interacting with the Medley Forma frontend (represented by the "Medley Forma Frontend" node). The user submits a form through the frontend interface.

The Medley pharma frontend then sends a request to the Medley Forma backend (represented by the "Medley Forma Backend" node). The backend processes the request and the associated form data.

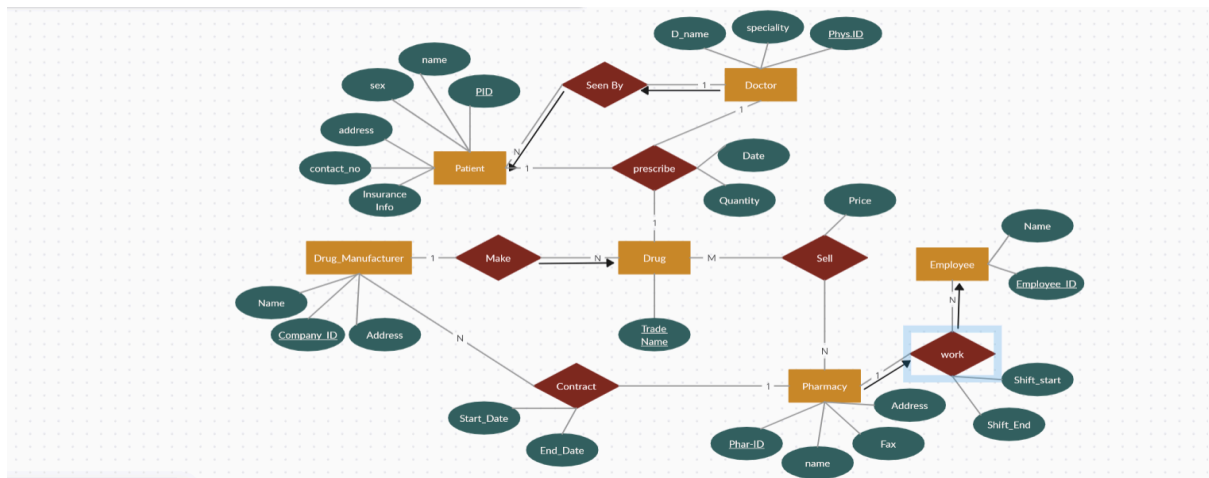
The backend communicates with a database (represented by the "Database" node) to store the form submissions securely. The database stores the submitted form data, including the responses provided by the user.

Additionally, the backend may utilize a file storage system (represented by the "File Storage" node) to store any files or attachments associated with the form submissions. This can include uploaded images, documents, or any other files relevant to the form.

Once the backend has processed the request, it sends a response back to the Medley Formafrontend. The frontend then displays the form response to the user.

The "style" statements at the bottom are optional and can be used to customize the appearance of specific nodes. In this example, the "User" and "File Storage" nodes have a light gray background.

ER DIAGRAM:



The Entity-Relationship (ER) diagram for Medly Pharma represents the various entities and their relationships within the pharmaceutical management system. Here is a description of the key components of the ER diagram:

1. Entities:
 - Product: Represents pharmaceutical products available in the system, with attributes such as product ID, name, description, and price.
 - Customer: Represents customers who purchase pharmaceutical products, with attributes such as customer ID, name, address, and contact information.
 - Order: Represents orders placed by customers, with attributes such as order ID, date, total amount, and status.
 - Sales Representative: Represents sales representatives who promote and sell pharmaceutical products, with attributes such as sales rep ID, name, and contact information.
 - Supplier: Represents suppliers who provide pharmaceutical products to the system, with attributes such as supplier ID, name, and contact information.
2. Relationships:
 - Product Order: Represents the relationship between products and orders, indicating which products are included in each order.
 - Customer Order: Represents the relationship between customers and orders, indicating which orders are placed by each customer.
 - Sales Rep Order: Represents the relationship between sales representatives and orders, indicating which sales representatives are responsible for each order.
 - Product Supplier: Represents the relationship between products and suppliers, indicating which suppliers provide each product.

3. Attributes:

- Each entity has its own set of attributes that describe its properties. For example, the Product entity has attributes such as product ID, name, description, and price.

4. Primary Keys:

- Each entity has a primary key that uniquely identifies each record in the entity. For example, the Product entity's primary key is the product ID.

5. Foreign Keys:

- Foreign keys are used to establish relationships between entities. For example, the Order entity may have a foreign key that references the customer ID to link orders to customers.

Overall, the ER diagram for Medly Pharma provides a visual representation of the database schema, showing how different entities are related and how data flows between them in the pharmaceutical management system.

Key Features:

The key features of a MedleyPharma booking app can vary depending on its specific functionality and target audience. However, here are some common key features that a booking app like MedleyPharma might include. **User Registration and Profiles:** Allow users to create accounts, set up profiles, and manage their personal information.

Booking Management: Allow users to search, browse, and book services based on their preferences, including selecting specific dates, times, and service providers.

Payment Integration: Facilitate secure online payments, supporting various payment methods, such as credit/debit cards, mobile wallets, or digital payment platforms.

Reviews and Ratings: Enable users to provide feedback and rate their experience with the booked services, helping others make informed decisions.

Booking History: Maintain a record of past and upcoming bookings for users to reference and manage. Remember, these are just general features that a booking app like Medley Pharma might have. The actual feature set can be customized based on the specific needs and requirements of the platform.

ROLES AND RESPONSIBILITIES:

User:

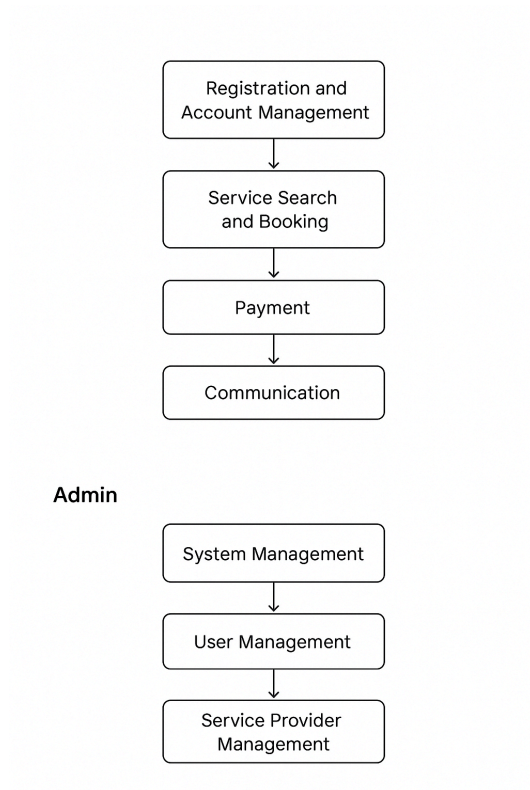
- **Registration and Account Management:** Customers are responsible for creating an account, providing accurate personal information, and managing their account details, including profile updates and password security.
- **Service Search and Booking:** Customers can search for available services based on their preferences, such as location, date, time, and type of service. They are responsible for selecting the desired service, booking it, and providing any necessary information for the booking.
- **Payment:** Customers are responsible for making payments for the booked services through the available payment methods provided by the app. They should ensure the accuracy and security of their payment details.
- **Communication:** Customers may need to communicate with service providers or the app's customer support team for inquiries, changes, or cancellations related to their bookings. They should utilize the provided communication channels appropriately and adhere to any specified guidelines.
- **Feedback and Reviews:** Customers have the responsibility to provide honest and constructive feedback on the services they have availed. They can submit reviews, ratings, and comments to help other users make informed decisions.

Admin :-

- **System Management:** The admin is responsible for overall management and administration of the Medley Forma booking app. This includes monitoring system performance, ensuring data security and privacy, and managing app settings and configurations.
- **User Management:** The admin is responsible for managing user accounts, including registration, verification, and account moderation. They may handle tasks such as account approvals, suspensions, or terminations when necessary.
- **Service Provider Management:** The admin oversees the registration, verification, and approval process for service providers who offer services.

through the app. They ensure the accuracy and quality of the service providers listed on the platform.

User Flow:



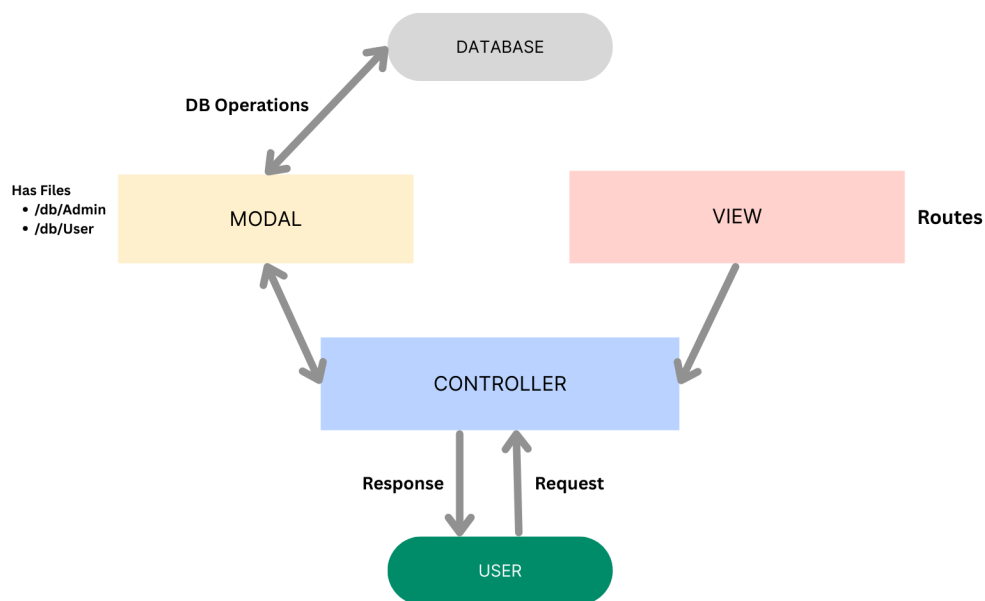
Customer Flow

- **Registration & Account** – Create and manage account details securely.
- **Search & Booking** – Find services by filters and complete bookings.
- **Payment** – Pay securely using available methods.
- **Communication** – Contact providers/support for queries or changes.
- **Feedback** – Leave honest reviews to help improve services.

Admin Flow

- **System Management** – Oversee app performance, settings, and security.
- **User Management** – Verify, approve, suspend, or manage user accounts.
- **Service Provider Management** – Verify and maintain quality of service providers.

MVC PATTERN:



The **MedlyPharma** application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The **Controller** layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

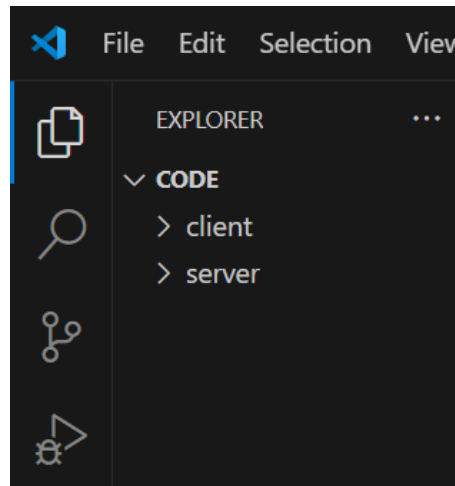
In the context of a backend REST API, the **View** is implemented as the **routing layer**, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New features can be added easily by creating new routes, controllers, and models.
- **Reusability:** Logic in controllers and models can be reused across multiple parts of the application.
- **Testing:** Each layer can be tested independently, especially the controllers and models.
- **Collaboration-Friendly:** Multiple developers can work simultaneously on different layers without conflict.

Project Setup And Configuration.

- **Creating project folder**
 1. Create a new folder with your <project name>.
 2. Inside that folder create two new folders.
 3. Name one as ****Client****.
 4. Name another one as ****Server****.
 5. Now open that folder in VS Code.



- **Client setup (installing react app)**

- Open the Client folder in the terminal of VScode.
 - `npm create vite@latest . -- --template react`
- Select React framework from the given options.
 - Select a framework:
 - | React
- Select JavaScript variant from the given options.
 - Select a variant:
 - | JavaScript
- Now lets navigate to the client folder by giving the following command.
 - `cd client`
- To install all the packages run the following command.
 - `npm install`
- To start the React server type the following command.
 - `npm run dev`

- **Server setup (npm init)**

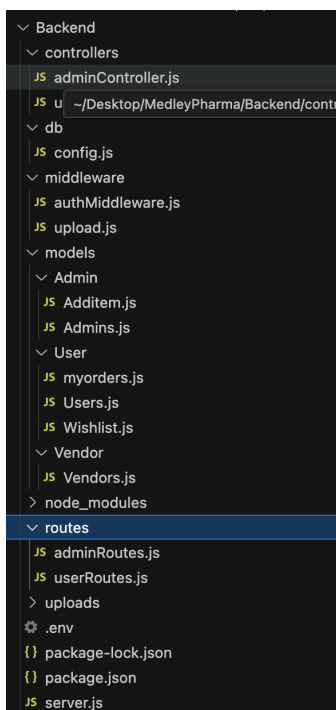
- Open Server folder in terminal of VScode.
 - `npm init -y`
- Create files:
 - `server.js`
- Create folders:
 - `models`
 - `controllers`
 - `Routes`

BACKEND DEVELOPMENT:

Backend Reference Video:

https://drive.google.com/file/d/1nUiTOVe3hfDy3R8zSDpZFaQfn4qpHOde/view?usp=drive_link

Backend Structure:



Controllers

- **adminController.js** → Handles all admin-related business logic (login, manage vendors, items, etc.).
- **userController.js** → Handles all user-related operations (register, login, orders, wishlist, etc.).

db

- **config.js** → Contains database connection setup (e.g., MongoDB/SQL config).

middleware

- **authMiddleware.js** → Middleware for authentication/authorization (JWT/session validation).
- **upload.js** → Middleware for handling file uploads (e.g., product images, profile pics).

models

- **Admin**
 - **Additem.js** → Schema/model for items added by admin (products, medicines, etc.).
 - **Admins.js** → Schema/model for admin accounts.
- **User**
 - **myorders.js** → Schema/model for storing user orders.
 - **Users.js** → Schema/model for user accounts.
 - **Wishlist.js** → Schema/model for user wishlist items.
- **Vendor**
 - **Vendors.js** → Schema/model for vendor accounts and details.

routes

- **adminRoutes.js** → Defines API routes for admin actions (login, add items, approve vendors, etc.).
- **userRoutes.js** → Defines API routes for user actions (register, login, place order, wishlist, etc.).

DATABASE DEVELOPMENT:

1. Configure MongoDB:

- Install Mongoose.
In your Node.js project directory, run:
`npm install mongoose`
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

```
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
  } catch (error) {
    process.exit(1);
  }
};

module.exports = connectDB;
```

Create Schemas:

Firstly, configure the Schemas for MongoDB database, to store the data in such a pattern. Use the data from the ER diagrams to create the schemas. The Schemas for this application look alike to the one provided below.

1) **Users.js** → Schema/model for user accounts.

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  name:String,
  email: String,
  password: String,
  userId:{
    type:mongoose.Schema.Types.ObjectId,
    ref:"user",
  }
})

module.exports =mongoose.model('user',UserSchema)
```

2) **myorders.js** → Schema/model for storing user orders.

```
const mongoose = require('mongoose');

const orderschema = new mongoose.Schema({
  flatno:String,
  pincode:String,
  city:String,
  state:String,
  totalamount:String,
  seller:String,
  sellerId:String,
  itemname:String,
  itemImage:String,
  description:String,
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  userName:String,
  BookingDate: {
    type: String, // Store dates as strings
    default: () => new Date().toLocaleDateString('hi-IN') // Set the default value
  },
  Delivery: {
    type: String, // Store dates as strings
    default: () => {
      // Set the default value to the current date + 7 days in "MM/DD/YYYY" format
      const currentDate = new Date();
      currentDate.setDate(currentDate.getDate() + 7); // Add 7 days
      const day = currentDate.getDate();
      const month = currentDate.getMonth() + 1; // Month is zero-based, so add 1
      const year = currentDate.getFullYear();

      // Format the date in "MM/DD/YYYY" format
      const formattedDate = `${month}/${day}/${year}`;

      return formattedDate;
    }
  }
})
```

3) **Wishlist.js** → Schema/model for user wishlist items.

```
const mongoose = require('mongoose');

const carschema = new mongoose.Schema({
  user_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
  },
  item_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Item',
  },
});

module.exports = mongoose.model('wishlisteditems', carschema)
```

4) **Vendors.js** → Schema/model for vendor accounts and details.

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  name:String,
  email: String,
  password: String,
  userId:{
    type:mongoose.Schema.Types.ObjectId,
    ref:"vendor",
  }
})

module.exports =mongoose.model('vendor',UserSchema)
```

5) **Additem.js** → Schema/model for items added by admin (products, medicines, etc.).

```
const mongoose = require('mongoose');

const itemschema = new mongoose.Schema({
  itemImage:String,
  description:String,
  itemtype:String,
  price:String,
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  userName:String,
})

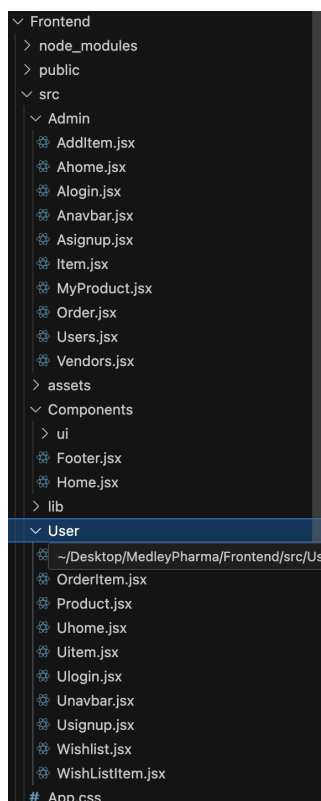
module.exports =mongoose.model('items',itemschema)
```

FRONT-END DEVELOPMENT:

Frontend Reference Video:

https://drive.google.com/file/d/106rDdPt2wgxXbatSxbErE16jnOs1qf3t/view?usp=drive_link

Frontend Structure:



Admin (src/Admin)

- **AddItem.jsx** → UI for admin to add new products/services.
- **Ahome.jsx** → Admin dashboard/homepage.
- **Alogin.jsx** → Admin login page.
- **Anavbar.jsx** → Navigation bar for admin panel.
- **Asignup.jsx** → Admin signup/registration page.

- **Item.jsx** → Component to display/manage individual product/service.
 - **MyProduct.jsx** → Shows products/services added by the admin.
 - **Order.jsx** → Admin view for managing customer orders.
 - **Users.jsx** → Admin panel to manage registered users.
 - **Vendors.jsx** → Admin panel to manage vendors/service providers.
-

User (src/User)

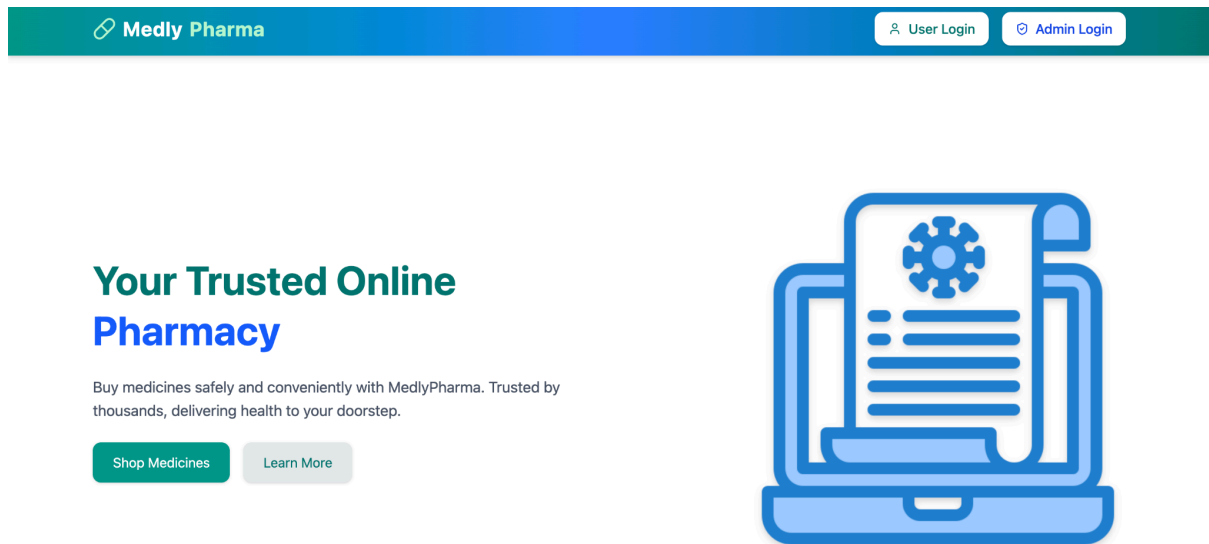
- **OrderItem.jsx** → Displays details of an individual order for the user.
- **Product.jsx** → Shows product/service details for users.
- **Uhome.jsx** → User homepage/dashboard.
- **Uitem.jsx** → Component to display a single product/service to the user.
- **Ulogin.jsx** → User login page.
- **Unavbar.jsx** → Navigation bar for user side.
- **Usignup.jsx** → User signup/registration page.
- **Wishlist.jsx** → Displays all wishlist items of a user.
- **WishListItem.jsx** → Component for a single wishlist item.

Components (Reusable parts)

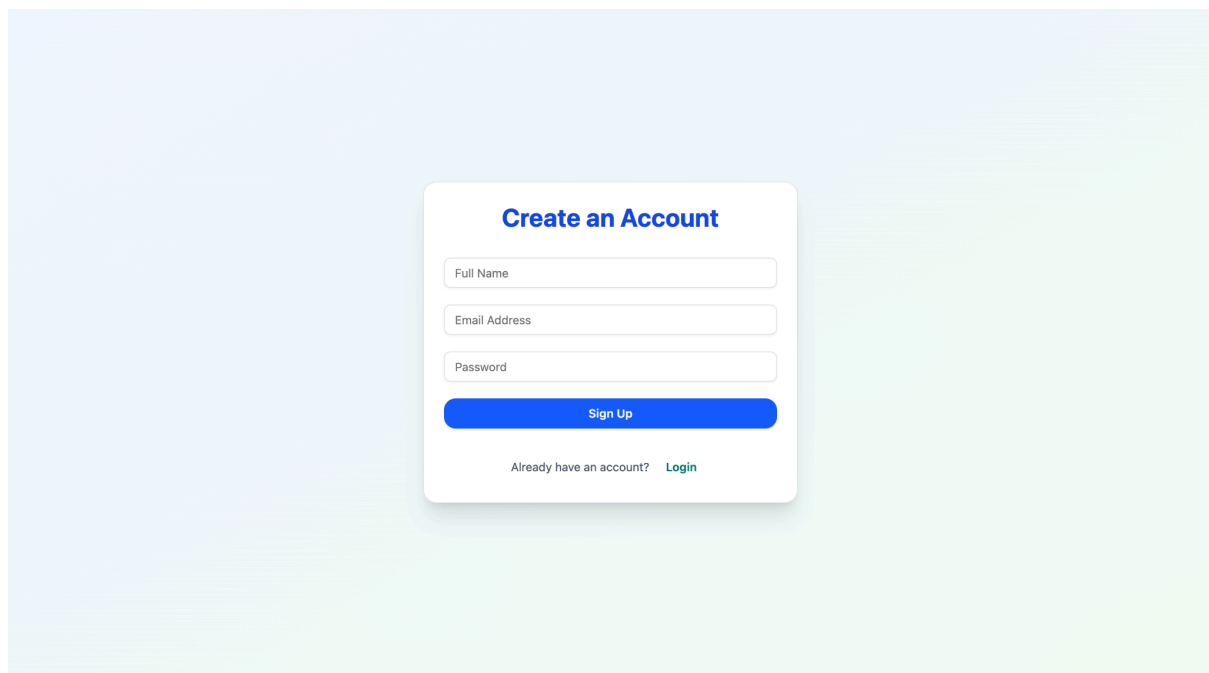
- **Footer.jsx** → Footer component used across pages.
- **Home.jsx** → Main home/landing page (probably entry page for app).
- **ui/** → Likely contains shared UI components (buttons, modals, etc.).

Output ScreenShots:

Landing Page:



Register.jsx(Admin):



Login.jsx:

Admin Login

Email Address

Enter your email

Password

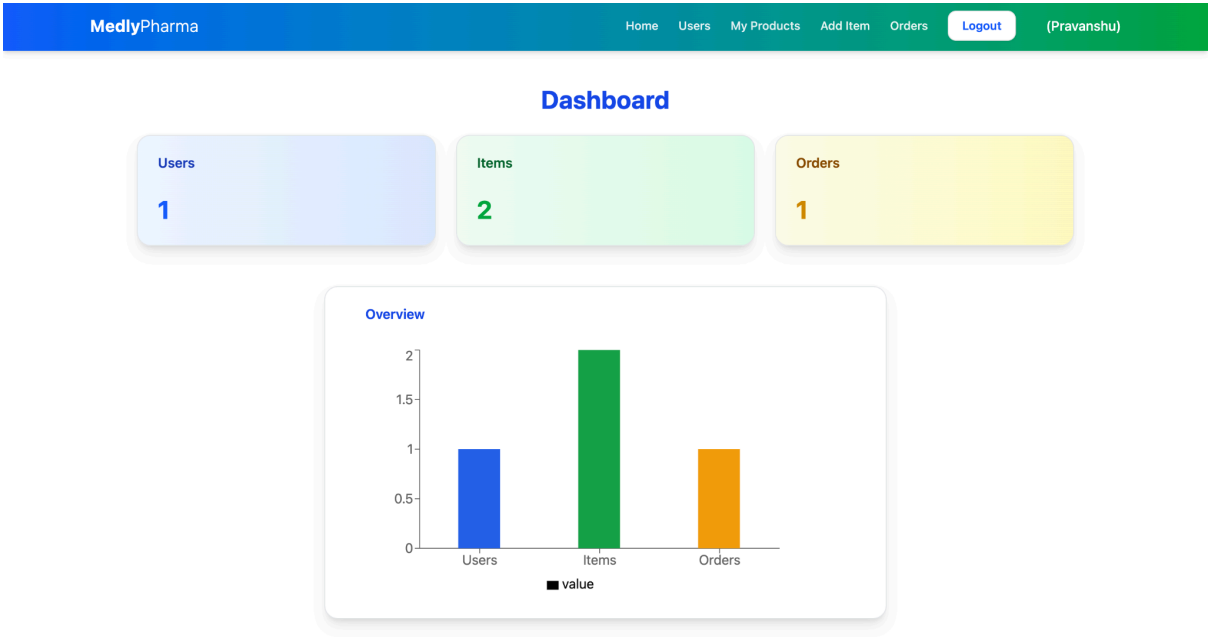
Enter your password

Log In

Don't have an account?

Create an Account

Admin HomePage:



User’s Page:

MedlyPharma

Home

Users

My Products

Add Item

Orders

Logout

(Pravanshu)

Users

All Registered Users

#	User ID	Name	Email	Actions
1	68c3c99a15383e499e964ca5	Abhi	abhi@gmail.com	View Orders

A list of registered users in the system

MyProductPage:

MedlyPharma

Home

Users

My Products

Add Item

Orders

Logout

(Pravanshu)

Medicines List

Search by Name

Search under Price

Sort Price Low → High

Calpol – 3e2e

₹22

View

Paracetamol 500mg Tablet – 3e14

₹25

View

Contact Us

AddItemPage:

MedlyPharma

Home

Users

My Products

Add Item

Orders

Logout

(Pravanshu)

Add Item

Item Name

Enter Item Name

Price

Enter Price

Description

Enter Description

Item Image

Choose File No file chosen

Submit

Order's Page:

MedlyPharma

Home

Users

My Products

Add Item

Orders

Logout

(Pravanshu)

Orders

Order #68c3e744

On the Way

Paracetamol 500mg Tablet

Customer

Abhi

Address

vvhrbv, r3rwfr, rjbj - 378543

Booking Date

12/9/2025

Delivery By

9/19/2025

Price

₹54

Contact Us

Medly-Pharma is your complete solution for online medicine ordering.

📞

Call At: XXXXXXXXXX

© 2025 Medly-Pharma.

All Rights Reserved.

User’s Registration Page:

Register

Full Name

Email Address

Password

Sign Up

Already have an account? [Login](#)

User’s Login Page:

User Login

Email Address

Enter your email

Password

Enter your password

Log In

Don't have an account? [Create an Account](#)

User’s HomePage:

Medly Pharma

HomeProductsMy Orders

(Abhi)Logout

Meddly - Pharma

Top Medicines At Best Price

Our Medicines

Medicines



User’s Products Page:

Medly Pharma

HomeProductsMy Orders

(Abhi)Logout

Medicine List

Search by Medicine Name

Search under Price

Low to High

Calpol

Price: ₹22

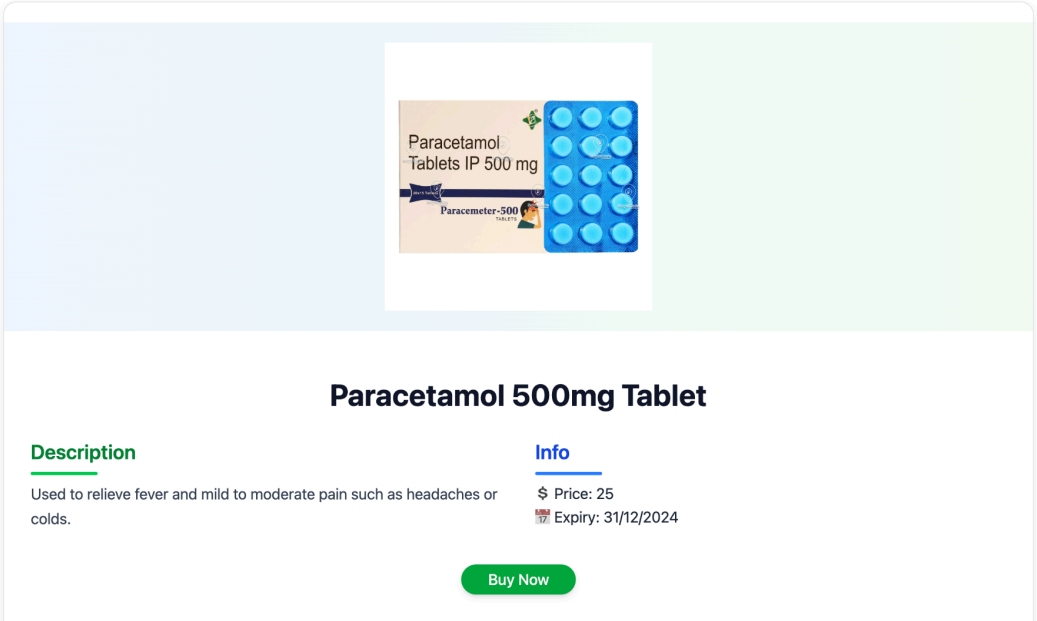
View Details

Paracetamol 500mg Tablet

Price: ₹25

View Details

Downloaded from <http://ajphaphysocpubs.phapublications.org/> on November 10, 2015



Downloaded from <http://ajph.org/> on November 10, 2015

Order’s Tracking Page:

Medly Pharma

HomeProductsMy Orders

(Abhi)Logout

My Orders



Paracetamol 500mg Tablet - #3e74

Order ID: 68c3e7444f

Address

vvhrbv, r3rwfr (378543), rjbj

Seller

Pravanshu

Booking Date

12/9/2025

Delivery By

9/19/2025

Warranty

1 year

Price

₹54

Status

On the way

Contact Us

Medly-Pharma is your complete solution for online medicine ordering.

Call At: XXXXXXXXXX

© 2025 Medly-Pharma.

All Rights Reserved.

Demo Video Link:

https://drive.google.com/file/d/1CxNV5bC6C2DkFNV-onBB-P61Qr9j6BLi/view?usp=drive_link

Code Drive Link:

https://drive.google.com/drive/folders/1pq8wmzlArK_KnFJkd1dbBvQQcYao3l6v?usp=drive_link